



DD2477 Project

Searching the Scientific Literature

Georges Tohme
tohme@kth.com

Annysia Dupaya
dupaya@kth.com

Gustaw Siedlarski Roger Vendrell Colet
gustawsi@kth.com rogervc@kth.com

May 7, 2025

Abstract

Research publication portals often rely on exact keyword matches for search, making it difficult for users to find the papers they need for their specific research interest. We firstly implement semantic search in an *Elasticsearch* instance, and then extend it to an author-personalized approach, which reranks the retrieved documents of the semantic search according to the author's research interests. We build a search engine for a subset of the scientific papers published by KTH researchers in DiVA, specifically only those in the Computer Science field. We have found that both semantic search and personalized search usually outperform exact keyword matching. However, general semantic search can sometimes perform better than the extended personalized search, depending on the level of detail (or ambiguity) of the query. Future work using real user judgement data could help create a better reranking model.

Contents

1	Introduction	3
2	Related Work	3
3	Methods	3
3.1	Standard Search	3
3.1.1	Semantic Search	3
3.2	Personalized Search	4
3.2.1	Topic Assignment	4
3.2.2	Coauthor Graph	5
3.2.3	Query Generation	6
3.2.4	Model training	6
4	Evaluation	7
5	Results	8
6	Discussion	8

1 Introduction

With the ever-increasing volume of scientific literature being published every day, the difficulty of finding papers relevant to a user’s needs is also ever-increasing. The traditional search methods typically used by scientific literature databases heavily depend on exact keyword matches, which often fail to grasp the intent of user queries, rendering them insufficient. This project aims to create and evaluate a semantic embedding and user-profile based search system to improve the efficacy of information retrieval on scientific literature. Particularly, we aim to enhance the searching process for existing authors, building a user profile for them based on their past publications. Our use case focuses on the Computer Science subset of scientific literature published by researchers from KTH. The data is available in the DiVA archive of scientific literature.

2 Related Work

The usage of general purpose sentence transformer models as a part of search engine systems is a relatively new innovation in the world of information retrieval, with much of the research in the field being from the past 3 years and the earliest research we could find within this area being from 2019 [8]; which had promising results showing a general purpose multilingual transformer model outperforming other word embedding models on information retrieval tasks. Previous attempts at incorporating general purpose sentence transformer models into scientific literature search systems, particularly in the medical field, have found significant improvements in the quality and utility of transformer based search over traditional search systems [1, 6].

3 Methods

3.1 Standard Search

A standard lexical search option was implemented using *Elasticsearch* to serve as a baseline for comparison with the newly developed sentence transformer-based search system. The system utilizes the titles, authors, and abstracts of the indexed research papers to retrieve results relevant to the input queries.

3.1.1 Semantic Search

The semantic search option was implemented by applying a sentence transformer model (`all-mpnet-base-v2`) on the abstracts of research papers during the indexing stage and storing the resulting vector representations in an *Elasticsearch* index as a `dense_vector` type, searchable with an L2-norm similarity metric. At query time, a vector embedding of the search query is created using the same sentence transformer model, and a k nearest neighbor (kNN) search is performed on the indexed vector representations to find and rank relevant documents.

3.2 Personalized Search

The article by Jakob Max on personalizing search through learning-to-rank with *Elasticsearch* [5] adds user preferences to search queries in order to personalize results. We simulate user preference and interactions by assigning topics to the papers published by every author in the dataset, therefore quantifying which topics each author mostly engages with. This helps building a model to rerank the documents retrieved by the general search. Additionally, we assess how closely the querying user is to the authors of the retrieved documents (in terms of collaboration distance), which also helps refining the reranking process.

With this information, we train a model which, after receiving the query and user preferences, re-scores the items from our general/semantic search to fit the user’s information need. The following subsections define this process in more detail.

3.2.1 Topic Assignment

To enhance the relevance of search results based on user preferences, we begin by assigning topics to each paper in our dataset. The goal of this process is to classify each document into a set of predefined categories that reflect its main subjects. The topics include categories such as "Machine Learning / Deep Learning", "Computer Vision / 3D", "Natural Language Processing", and many others (see Figure 1 for the full list). These categories are chosen to represent the primary areas of research within the Computer Science field.

For this task, we use a zero-shot classification model (`facebook/bart-large-mnli` [4] pre-trained Sentence Transformer from Hugging Face) to assign topics to the papers. Zero-shot classification allows us to classify documents into categories without requiring task-specific training data. The model encodes each paper’s title, abstract and keywords (joined together as one single string) into a high-dimensional vector representation. It does the same for each of the topic categories, representing them in the same high-dimensional space. The model then evaluates the semantic similarity between each paper’s encoding and the set of encodings of the predefined topic categories. This yields, for each scientific paper, a vector of probabilities of belonging to each topic. Figure 1 shows an example of the result of this process, the topic probabilities assigned to the paper *Visual Debugging of Dataflow Systems*.

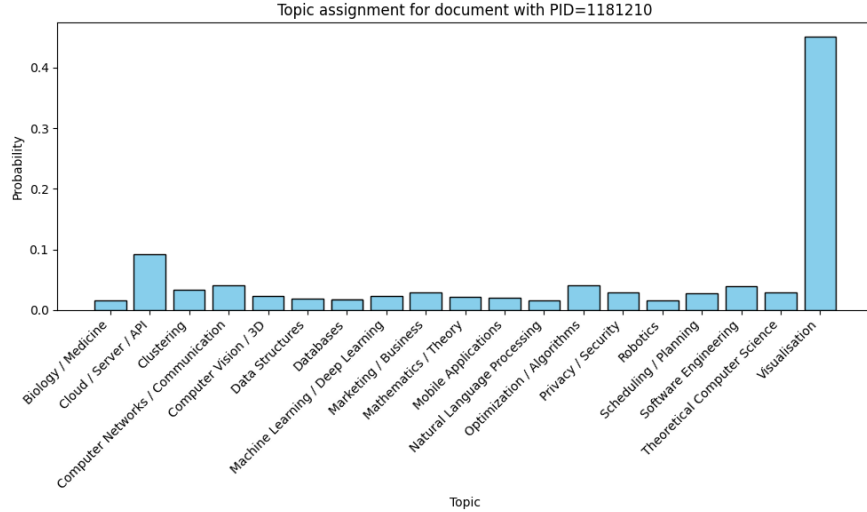


Figure 1: Topic probabilities assigned to the paper *Visual Debugging of Dataflow Systems*.

Once we are able to determine topic probabilities for scientific papers, we compute the average of the topic-probability vectors of the articles each author has published in the past. This averaged vector carries the information about the research interests of the author, and is stored in a separate index to serve as a user profile.

3.2.2 Coauthor Graph

The coauthor graph serves as a representation of the collaborative relationships between authors, based on their shared publications in the dataset. Each node in the graph represents an individual author, while the edges between nodes represent coauthorship of a paper.

We assume that authors who are closely connected in the coauthor graph are more likely to share similar research interests or work in related fields. This assumption is supported by [7], which demonstrates that academic collaboration networks exhibit high-clustering and small-world properties (indicating that authors tend to collaborate within distinct communities which are defined by their topics of interest).

By using this structure, we can improve search result relevance by giving higher priority to papers authored by researchers connected to the querying author. In our reranking process, we not only consider direct coauthors but also those up to three degrees of separation (that is, coauthors of coauthors and their collaborators) to capture a broader but still meaningful range of related work.

Figure 2 shows a visual representation of the coauthor graph. The subfigure 2a shows the whole graph (note that the visualization only shows authors with at least one collaboration, even though our data set also comprises authors without collaborations). Authors with exactly one collaboration (that is, isolated pairs of authors) are portrayed mostly in an outer ring, which is an arbitrary layout that lets us focus on the more important part of the graph: the collaboration clusters (seen in the middle). Subfigure 2b is a zoom-in into the center of Subfigure 2a, the clusters, and confirms the properties assumed regarding academic collaboration. Subfigure 2c, a zoom-in into one of the clusters in Subfigure 2b, confirms that

taking into account other authors up to a collaboration distance of 3 is enough to grasp most of the related papers in a cluster.

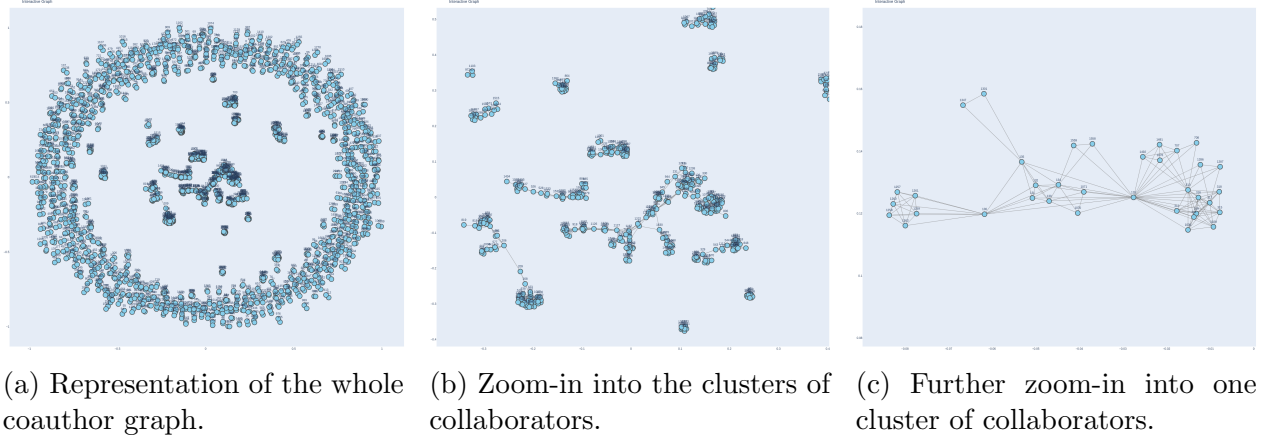


Figure 2: Coauthor graph visualisation.

3.2.3 Query Generation

To substitute for real-life user input (meaning, to simulate real users interacting with our search engine), we use the coauthor graph, keywords associated with the papers (as provided in the original dataset from DiVA) and their author to create queries and assign a score to each.

We created a new table of authors, their works, and keywords of said works. For each author, we create one query based for every keyword in their papers. The relevance score of the given articles is based on the distance between the two authors, with 3 being the most relevant (authors have worked together before) and 0 least relevant (no relation between the two authors). This is meant to simulate user interactions similar to the shopping cart example in the Hopsworks paper on the two-tower embedding model [3] [2].

3.2.4 Model training

The methodology for training the reranking model is based on Jakob’s recommendations in his article with *Elasticsearch* Labs [5]. We use the data from both documents and generated queries (also referred to as ”judgements”). From the previous activities, we also add another column ”Topic” to the document information, which represents the highest scoring topic for that document. We used XGBoost as the learning-to-rank rescorer supports XGBRanker trained models.

We one-hot encoded each topic as a feature in the machine while also encoding each topic with the user’s preference to the topic, as well as the user’s preferences for each topic. Figure 3 shows the importance of each feature to the model. In general, after the topic of each document, user preferences are the most important, followed by the topic each paper is categorized under.

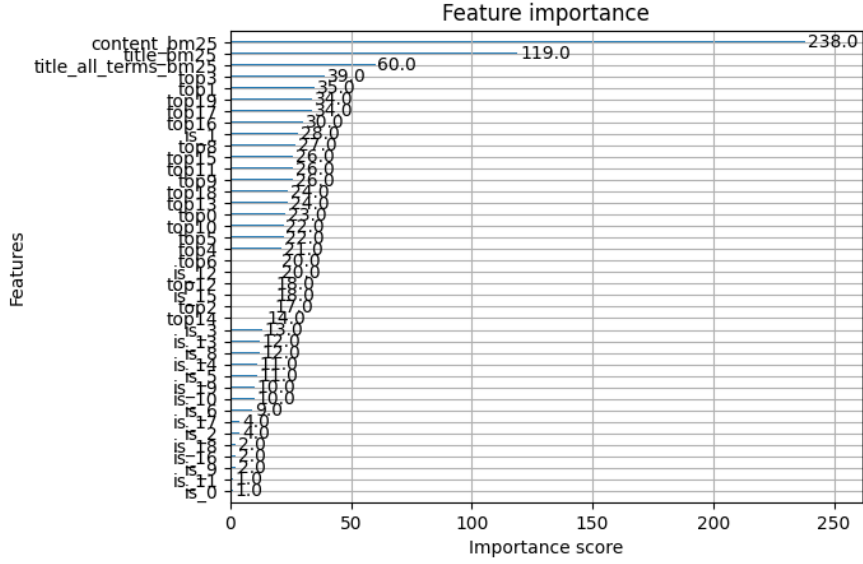


Figure 3: Importance of each feature to the rescoring model, where **content** is the abstract of the paper, **title** its title, **top_i** is the i-th topic and **is_i** is the one-hot encoding of the topic with the highest probability.

After training the reranking model, our local *Elasticsearch* instance can then use it to rescore the items based on user preferences. We first retrieve user preferences from its respective index, and add it to the rescore parameters.

4 Evaluation

To evaluate the results, we selected a number of authors and created a search query based on the author’s topics of interest. We then computed the NDCG@50 from the results of each query.

$$NDCG@K = \frac{DCG@K}{IDCG@K} = \frac{\sum_i^{k \text{ (actual order)}} \frac{rel_i}{\log_2(i+1)}}{\sum_i^{k \text{ (ideal order)}} \frac{rel_i}{\log_2(i+1)}} \quad (1)$$

We selected authors with several publications in the dataset, allowing us to infer their research interests. We created queries based on their works, and ranked each result from 0-3 (3 as the most relevant). To avoid introducing bias in our manual assessment, we collected the top 50 retrieved documents by each ranking method (keyword-match, semantic or personalized), and shuffled them. After assessing the relevance of each document, we compute the NDCG@50, which can be seen in Section 5.

5 Results

The results show that lexical search performs worst on average among all three search types. Only if the queries happen to be exact matches of popular paper keywords and seem particularly niche for the dataset, they work best (such as in the query *sensor integration techniques*). For more vague search queries such as "pickup and delivery", or "modeling new environments", personalized recommendations perform better, showing that reranking of the results given by the semantic model is useful. More specific queries (ex. "integrating CNNs with robotics systems") seem to not benefit from user profiles, but rather the opposite: if the user is already looking for specific content, tailoring results to their user profile might add unwanted noise.

Query	Lexical	Semantic	Personalized
<i>high performance networking algorithms</i>	0.16	0.84	0.59
<i>efficient pickup and delivery</i>	0.52	0.78	0.81
<i>integrating CNNs with robotics systems</i>	0.21	0.67	0.51
<i>modeling new environments</i>	0.00	0.34	0.52
<i>machine learning for deliveries</i>	0.34	0.71	0.79
<i>sensor integration techniques</i>	0.35	0.19	0.20
<i>obstacle prediction</i>	0.44	0.55	0.66

Table 1: NDCG@50 values for a set of assessed queries on the 3 implemented retrieval systems.

6 Discussion

We modeled user preferences by averaging topic probabilities across each author’s previous publications, using zero-shot classification to assign topics without requiring labeled training data. This approach enabled us to personalize search results based on inferred research interests, enhancing retrieval relevance for general queries. The coauthor graph further enriched personalization by factoring in collaborative relationships, boosting the visibility of research from related (though indirect) authors.

Based on the NDCG results, we found the semantic and personalized search options perform well on queries that do not necessarily match the scientific paper’s content word-wise. Specifically, semantic search benefits from increased specificity in a query, while personalized search is useful to tailor a rather general query to the user’s known interests. However, even though, the NDCG scores we gathered show promise (with relevant results for each user ranked near the top) they are still synthetic results and are not statistically rigorous. Future work could be improved by having a more robust model, which could be made possible with more resources, such as having real user interactions from which to base the training on.

References

- [1] ASMALA, S. Improving the search of medical articles using sentence transformers. [*dipl*] *Perustieteiden korkeakoulu / SCI* (May 2024).
- [2] DE LA RÚA MARTÍNEZ, J., BUSO, F., KOUZOUPIS, A., ORMENISAN, A. A., NIAZI, S., BZHALAVA, D., MAK, K., JOUFFREY, V., RONSTRÖM, M., CUNNINGHAM, R., ET AL. The hopsworks feature store for machine learning. In *Companion of the 2024 International Conference on Management of Data* (2024), pp. 135–147.
- [3] HOPSWORKS. What is a two-tower embedding model?
- [4] HUGGINGFACE. facebook/bart-large-mnli.
- [5] JAKOB, M. Personalized search with elasticsearch learning to rank (ltr), Nov 2024.
- [6] JOSE, A., AND HARIKUMAR, S. Covid-19 semantic search engine using sentence-transformer models. In *Computational Intelligence, Cyber Security and Computational Models. Recent Trends in Computational Models, Intelligent and Secure Systems* (Cham, 2022), I. Raman, P. Ganesan, V. Sureshkumar, and L. Ranganathan, Eds., Springer International Publishing, pp. 189–200.
- [7] NEWMAN, M. E. J. The structure of scientific collaboration networks. *Proceedings of the National Academy of Sciences* 98, 2 (Jan. 2001), 404–409.
- [8] YANG, Y., CER, D., AHMAD, A., GUO, M., LAW, J., CONSTANT, N., ABREGO, G. H., YUAN, S., TAR, C., SUNG, Y.-H., STROPE, B., AND KURZWEIL, R. Multilingual universal sentence encoder for semantic retrieval, 2019.